DMC AI Trading Agent

A High-Fidelity Market Simulation and Curriculum-Based Reinforcement Learning Framework

Executive Whitepaper (Public Lite Version)

Yu Ling Sun

March 2026

Abstract

This document specifies the core design and implementation architecture for the DMC AI Trading Agent. Rather than predicting future prices or optimizing static strategies, the system focuses on decision-making quality, execution realism, and progressive skill acquisition. A high-fidelity market simulation environment is constructed using authentic exchange mechanics, while a curriculum learning framework incrementally introduces market complexity. The architecture emphasizes modularity, decoupling cognitive market reading from high-frequency execution reflexes, ensuring the resulting agent can operate in live markets with disciplined risk awareness and execution-consistent behavior.

Contents

1 Project Objectives and Environment Philosophy **3**

2 High-Fidelity Exchange Execution Simulator **3**

2.1 Trade Flow Simulation and Market Impact 3

2.2 Look-Ahead Bias Prevention 3

3 Data Architecture and Temporal Regimes **4**

3.1 Time Regime Partitioning 4

3.2 Multi-Scale Time Encoding 4

4 Hierarchical Attention-Memory (H-AM) Architecture **4**

4.1 The Manager: Strategic World Model (System 2) 5

4.2 The Worker: Tactical Execution Policy (System 1) 5

5 Curriculum Learning and Reward Design **5**

5.1 Phased Curriculum Framework 5

5.2 Evolutionary Reward Function 5

6 Exogenous Risk and Event-Driven Handling **6**

7 Conclusion **6**

1 Project Objectives and Environment Philosophy

This project aims to construct a realistic market simulation environment based on historical data and train an automated agent capable of human-like trading logic using Curriculum Learning and Reinforcement Learning (RL). The agent must understand multi-timeframe structures, liquidity, and order flow, producing behavioral decisions that are directly executable via exchange APIs.

The primary objective is not to optimize a fixed trading strategy, but to train a decision-making agent capable of reasoning over market structure and execution constraints. This project deliberately avoids conventional backtesting frameworks, opting instead for a custom RL environment grounded in the following principles:

- **State-Centric Design:** The environment is structured around evolving market states rather than predefined signal pipelines.
- **Decision-Level Learning:** The agent learns *when* and *how* to act under uncertainty.
- **Execution-Consistent Interface:** All actions are constrained to a realistic set of exchange-executable commands.
- **Progressive Fidelity:** Market realism is introduced incrementally through modular simulation layers.

2 High-Fidelity Exchange Execution Simulator

A custom execution engine was built to replicate realistic derivatives trading environments, eliminating common simplifications such as infinite liquidity or zero slippage.

2.1 Trade Flow Simulation and Market Impact

The simulator replaces computationally expensive Level-3 Order Book replay with a high-performance Trade Flow simulation that estimates liquidity constraints. Slippage is calculated using the **Square-Root Law of Market Impact**:

$$\text{Slippage} \approx c \cdot \sigma \cdot \sqrt{\frac{\text{Order Size}}{\text{Daily Volume}}} \tag{1}$$

where σ represents local volatility, and c is a dynamic impact coefficient that drastically increases during simulated liquidity droughts (e.g., liquidation cascades).

2.2 Look-Ahead Bias Prevention

To enforce strict temporal causality:

- **Tick-Driven Triggers:** Stop-loss (SL) and take-profit (TP) conditions are validated against the `aggTrades` stream.

- **Time-Gated Data Access:** The agent cannot access data beyond the current decision timestamp t.
- **Dynamic Margin:** The simulator faithfully replicates tiered margin mechanics and funding fee settlements.

3 Data Architecture and Temporal Regimes

To handle multi-year high-frequency data and multi-modal features, a high-performance data engineering stack is adopted, utilizing **Parquet** for columnar storage and **Polars** (Rust-based) for memory-safe feature computation.

3.1 Time Regime Partitioning

Historical data is explicitly divided into functional horizons to prevent look-ahead bias and align the training distribution with contemporary market conditions:

- **Context Horizon (2020–2023):** Accessible only as static, read-only information to establish structural regime exemplars. No trading actions or rewards occur here.
- **Training Horizon (2024 onward):** The period where the agent actively interacts, acquires skills via curriculum learning, and optimizes policy under realistic constraints.

3.2 Multi-Scale Time Encoding

The environment injects Temporal Context Embeddings to explicitly expose volatility expectations to the model.

Category	Feature Example	Data Type	Strategic Meaning
Price Action	`Log_Returns`, `Realized_Vol`	Float	Momentum & Volatility Intensity
Micro Time	`Hour_Sin/Cos`, `4H_Progress`	Float	UTC Time awareness (Session shifts)
Macro Time	`Dist_to_MonthlyOpen`	Float	Monthly macro regime positioning

Table 1: Selected Feature Engineering Metrics

4 Hierarchical Attention-Memory (H-AM) Architecture

To support the Cognition-Execution Decoupling strategy, the model adopts an H-AM framework. This design mimics biological cognitive systems by separating "System 2" (Slow, Deliberative) from "System 1" (Fast, Reflexive).

$$\text{Architecture} = \text{Worker}_{\text{PPO}}(\text{State}_{\text{Micro}}, \text{Goal}_{\text{Manager}}) \leftarrow \text{Manager}_{\text{Attention}}(\text{State}_{\text{Macro}}, \text{Memory}_{\text{Notebook}})$$

4.1 The Manager: Strategic World Model (System 2)

Active on a low-frequency heartbeat (e.g., 4H), the Manager interprets the market regime and sets strategic goals. It utilizes a Transformer-based Cross-Attention mechanism to query an External Dynamic Memory (**The Notebook**) that stores key structural levels (Support/Resistance).

$$\text{Context} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2}$$

By embedding temporal context into the Query (Q), the Manager learns to assign dynamic attention weights to structural levels based on the current trading session. It outputs a Goal Vector (g_t) defining directional bias and target aggression.

4.2 The Worker: Tactical Execution Policy (System 1)

Operating at high frequency (1-minute / Tick level), the Worker focuses on execution timing based on micro-structure signals such as **Order Flow Imbalance (OFI)** and price displacement velocity. It executes precise market entries and exits strictly within the Manager's strategic boundaries.

5 Curriculum Learning and Reward Design

The agent's learning trajectory mimics the developmental process of human traders, structured into four rigorous phases.

5.1 Phased Curriculum Framework

- **Phase 0 (The Observer):** The agent learns to map market structures and understand session awareness without assuming capital risk.
- **Phase 1 (Cruise):** The agent learns the "art of waiting," conserving resources during low-volatility periods.
- **Phase 2 (The Soldier):** Strict trend-following discipline. Counter-trend actions are heavily penalized.
- **Phase 3 (The Master):** Full execution authority unlocked, permitting high-frequency counter-trend execution (sniping) at validated liquidity zones.

5.2 Evolutionary Reward Function

The reward formulation evolves across curriculum phases. The objective function balances risk-adjusted returns, strategic alignment, and rule-based constraints:

$$R_t = w_1 R_{PnL} + w_2 R_{Bias} + w_3 R_{Valid} + w_4 R_{EP} - \text{Cost}_{transaction} \tag{3}$$

(Note: Specific weight matrices and structural penalty parameters are proprietary and dynamically adjusted during training.)

6 Exogenous Risk and Event-Driven Handling

This system treats extreme event-driven market regimes (e.g., macroeconomic news, black swans) as exogenous shocks. The DMC agent is optimized for standard market microstructures; therefore, event awareness is delegated to an external intelligence interface.

Risk Calibration Socket: At runtime, the system accepts a low-dimensional Risk State Packet via an Implicit Quantile Network (IQN) Gatekeeper. Typical gating behaviors include:

- Blocking new position openings while allowing risk reduction.
- Dynamically reducing allowable leverage constraints.
- Increasing simulated transaction costs to reflect anticipated adverse liquidity.

7 Conclusion

This framework presents a robust, execution-oriented pipeline for training an autonomous trading agent. By establishing the philosophies that **"perception precedes execution"** and **"risk awareness precedes capital allocation,"** the system successfully bridges the gap between academic reinforcement learning and practical quantitative execution.

The deliberate decoupling of core market logic from high-frequency execution reflexes protects the purity of early-stage learning, prevents overfitting to complex market noise, and preserves total flexibility for eventual live trading deployment.